

# Guia Operacional do Projeto Zabbix

Este guia mostra os comandos principais para executar, gerar relatorios, abrir arquivos, validar alteracoes e atualizar o GitHub.

Use este arquivo como referencia rapida quando quiser puxar um relatorio novo ou fazer ajustes pequenos no projeto.

## 1. Entrar na pasta do projeto

Execute sempre antes dos outros comandos:

```
cd /mnt/c/Users/chip/Documents/relat.incidentes.zabbix
```

Funcao:

- Coloca o terminal na pasta principal do projeto.
- Evita erro de caminho ao executar scripts.

## 2. Ativar o ambiente virtual

```
source zabbix-report/venv/bin/activate
```

Funcao:

- Ativa o Python isolado do projeto.
- Faz o terminal usar as bibliotecas instaladas na 'venv'.

Sinal de que deu certo:

```
(venv)
```

Para sair da venv:

```
deactivate
```

## 3. Testar conexao com o Zabbix

```
python zabbix-report/test_zabbix_api.py
```

Use quando:

- O relatorio falhar.
- O token do Zabbix for alterado.
- A API parecer lenta ou indisponivel.
- Voce quiser confirmar que o '.env' esta correto.

## 4. Comando principal do relatorio atual

Este e o comando mais importante do projeto:

```
python zabbix-report/zabbix_report.py --periodo historico --status abertos
```

Funcao:

- Busca todos os incidentes ainda abertos.
- Ignora incidentes ja resolvidos.
- Gera HTML, PDF e Excel atualizados.
- Mostra o estado operacional atual do Zabbix.

Arquivos gerados:

```
zabbix-report/reports/
```

Formatos:

- '.html': relatorio interativo com filtros.
- '.pdf': relatorio formal para envio.
- '.xlsx': planilha com os dados.

## 5. Gerar relatorio das ultimas 24h

```
python zabbix-report/zabbix_report.py --periodo 24h
```

Funcao:

- Busca eventos das ultimas 24 horas.
- Inclui abertos e resolvidos, salvo se voce usar '--status abertos'.

Somente abertos das ultimas 24h:

```
python zabbix-report/zabbix_report.py --periodo 24h --status abertos
```

## 6. Gerar relatorios por outros periodos

```
python zabbix-report/zabbix_report.py --periodo 2d
python zabbix-report/zabbix_report.py --periodo 5d
python zabbix-report/zabbix_report.py --periodo 7d
python zabbix-report/zabbix_report.py --periodo 30d
```

Funcao:

- Busca eventos dentro do intervalo informado.
- 'h' significa horas.
- 'd' significa dias.

## 7. Gerar relatorio desde uma data especifica

```
python zabbix-report/zabbix_report.py --desde 2026-06-01
```

Funcao:

- Busca eventos desde a data informada ate agora.
- Use o formato 'AAAA-MM-DD'.
- E util quando o historico completo estiver pesado.

## 8. Abrir o HTML gerado

Abra pelo Explorer:

```
C:\Users\chip\Documents\relat.incidentes.zabbix\zabbix-report\reports
```

Ou pelo terminal, ajustando o nome do arquivo:

```
cmd.exe /c start "" "C:\Users\chip\Documents\relat.incidentes.zabbix\zabbix-report\reports\report_2026-06-15_historico_abertos.html"
```

## 9. O que existe no HTML

O HTML atual possui:

- Resumo executivo.
- Filtro por unidade escolar.
- Filtro por equipamento.
- Filtro por severidade.
- Filtro por tempo offline.
- Prioridade operacional.
- Tipos de incidente consolidados.
- Hosts com mais incidentes.
- Unidades mais afetadas.
- Janela separada para Servidor Zabbix.
- Janela separada para CONFEE VPN.
- Exportacao CSV dos dados filtrados.

## 10. Tipos de incidente consolidados

O painel "Tipos de Incidente" nao deve contar cada texto tecnico isolado do

Zabbix. Ele consolida familias de problemas.

Exemplos:

- 'Unavailable by ICMP ping'
- 'High ICMP ping response time'
- 'High ICMP ping loss'
- 'No SNMP data collection'
- 'Temperature above threshold'
- 'High bandwidth usage'
- 'Ethernet lower speed'
- 'Interface down'

Arquivo onde isso e ajustado:

zabbix-report/classifiers.py

Funcao:

```
def classify_incident_type(incident):
```

Quando aparecer um novo texto do Zabbix que deveria entrar em uma familia existente, adicione uma regra nessa funcao.

## 11. Classificacao de equipamentos

Arquivo:

zabbix-report/classifiers.py

Funcao:

```
def classify_equipment(host):
```

Ordem operacional usada no filtro:

- Mikrotik
- Switch
- NVR
- Central de Alarme
- Terminal Facial
- Portal Detector de Metal
- Camera

Observacao:

- O painel "Equipamentos Mais Afetados" ordena por maior volume.
- O filtro "Equipamento" segue a ordem operacional acima.

## 12. Arquivos principais do projeto

zabbix-report/zabbix\_report.py

Coordena o fluxo principal: consulta Zabbix, processa dados, gera Excel, HTML e PDF.

zabbix-report/zabbix\_api.py

Centraliza chamadas para a API do Zabbix.

zabbix-report/classifiers.py

Guarda regras de classificacao de unidade, equipamento e tipo de incidente.

zabbix-report/summary.py

Calcula totais, rankings, indicadores, tempo offline e listas do resumo.

zabbix-report/templates/report\_template.html

Controla o visual e a interatividade do HTML.

```
zabbix-report/pdf_report.py
```

Gera o PDF operacional.

```
COMANDOS.md
```

Este guia.

## 13. Validar código depois de alterar

Sempre que mexer em Python:

```
python -m py_compile zabbix-report/*.py
```

Verificar espaços problemáticos no Git:

```
git diff --check
```

Ver resumo das mudanças:

```
git diff --stat
```

## 14. Ver arquivos alterados

```
git status
```

Função:

- Mostra arquivos modificados.
- Mostra arquivos novos.
- Mostra commits locais ainda não enviados.

## 15. Criar commit

Antes:

```
git status
```

Adicionar arquivos:

```
git add .
```

Criar commit:

```
git commit -m "mensagem clara do que mudou"
```

Exemplos:

```
git commit -m "feat: adiciona ranking de tipos de incidente"
git commit -m "fix: consolida tipos de incidente no ranking"
git commit -m "docs: atualiza guia operacional"
```

## 16. Enviar para o GitHub

```
git push origin main
```

Se pedir usuário:

```
lcmrcs
```

Se pedir senha:

```
cole o token do GitHub
```

Observação:

- O GitHub não aceita mais senha normal.
- Use Personal Access Token.
- Nunca cole token em arquivo do projeto.

## 17. Fazer pacote para supervisor

Arquivos recomendados:

- PDF: versao formal.
- HTML: versao interativa.
- XLSX: dados brutos.

Pasta usada anteriormente:

```
entrega_supervisor/
```

Essa pasta fica fora do Git pelo '.gitignore' da raiz.

## 18. Se o script ficar lento

Use um periodo menor:

```
python zabbix-report/zabbix_report.py --periodo 24h
```

Ou use uma data inicial:

```
python zabbix-report/zabbix_report.py --desde 2026-06-01
```

Para o estado operacional atual, prefira:

```
python zabbix-report/zabbix_report.py --periodo historico --status abertos
```

## 19. Se der timeout

Tente nesta ordem:

```
python zabbix-report/test_zabbix_api.py
python zabbix-report/zabbix_report.py --periodo 24h
python zabbix-report/zabbix_report.py --desde 2026-06-01 --status abertos
python zabbix-report/zabbix_report.py --periodo historico --status abertos
```

Se continuar falhando:

- Verifique internet/VPN.
- Verifique URL e token no '.env'.
- Aguarde alguns minutos e tente novamente.

## 20. Regra de seguranca

Nunca envie para o GitHub:

- '.env'
- token do Zabbix
- token do GitHub
- senhas
- arquivos temporarios
- pacotes de entrega com dados sensiveis

## 21. Fluxo rapido do dia a dia

Gerar relatorio atual:

```
cd /mnt/c/Users/chip/Documents/relat.incidentes.zabbix
source zabbix-report/venv/bin/activate
python zabbix-report/zabbix_report.py --periodo historico --status abertos
```

Validar alteracoes:

```
python -m py_compile zabbix-report/*.py
git diff --check
git status
```

Salvar e enviar:

```
git add .  
git commit -m "mensagem clara"  
git push origin main
```

## 22. Comando mais importante

Se voce esquecer todo o resto, lembre deste:

```
cd /mnt/c/Users/chip/Documents/relat.incidentes.zabbix  
source zabbix-report/venv/bin/activate  
python zabbix-report/zabbix_report.py --periodo historico --status abertos
```